

Data and Computation Transformations for Multiprocessors

Jennifer M. Anderson, Saman P. Amarasinghe and Monica S. Lam

Computer Systems Laboratory

Stanford University, CA 94305

Abstract

Effective memory hierarchy utilization is critical to the performance of modern multiprocessor architectures. We have developed the first compiler system that fully automatically parallelizes sequential programs and changes the original array layouts to improve memory system performance. Our optimization algorithm consists of two steps. The first step chooses the parallelization and computation assignment such that synchronization and data sharing are minimized. The second step then restructures the layout of the data in the shared address space with an algorithm that is based on a new data transformation framework. We ran our compiler on a set of application programs and measured their performance on the Stanford DASH multiprocessor. Our results show that the compiler can effectively optimize parallelism in conjunction with memory subsystem performance.

1 Introduction

In the last decade, microprocessor speeds have been steadily improving at a rate of 50% to 100% every year[16]. Meanwhile, memory access times have been improving at the rate of only 7% per year[16]. A common technique used to bridge this gap between processor and memory speeds is to employ one or more levels of caches. However, it has been notoriously difficult to use caches effectively for numeric applications. In fact, various past machines built for scientific computations such as the Cray C90, Cydrome Cydra-5 and the Multiflow Trace were all built without caches. Given that the processor-memory gap continues to widen, exploiting the memory hierarchy is critical to achieving high performance on modern architectures.

Recent work on code transformations to improve cache performance has been shown to improve uniprocessor system performance significantly[9, 34]. Making effective use of the memory hierarchy on multiprocessors is even more important to performance, but also more difficult. This is true for bus-based shared address space machines[11, 12], and even more so for scalable shared address space machines[8], such as the Stanford DASH multiprocessor[24], MIT ALEWIFE[1], Kendall Square's KSR-1[21], and the Convex

Exemplar. The memory on remote processors in these architectures constitutes yet another level in the memory hierarchy. The differences in access times between cache, local and remote memory can be very large. For example, on the DASH multiprocessor, the ratio of access times between the first-level cache, second-level cache, local memory, and remote memory is roughly 1:10:30:100. It is thus important to minimize the number of accesses to all the slower levels of the memory hierarchy.

1.1 Memory Hierarchy Issues

We first illustrate the issues involved in optimizing memory system performance on multiprocessors, and define the terms that are used in this paper. *True sharing* cache misses occur whenever two processors access the same data word. True sharing requires the processors involved to explicitly synchronize with each other to ensure program correctness. A computation is said to have *temporal locality* if it re-uses much of the data it has been accessing; programs with high temporal locality tend to have less true sharing. The amount of true sharing in the program is a critical factor for performance on multiprocessors; high levels of true sharing and synchronization can easily overwhelm the advantage of parallelism.

It is important to take synchronization and sharing into consideration when deciding on how to parallelize a loop nest and how to assign the iterations to processors. Consider the code shown in Figure 1(a). While all the iterations in the first two-deep loop nest can run in parallel, only the inner loop of the second loop nest is parallelizable. To minimize synchronization and sharing, we should also parallelize only the inner loop in the first loop nest. By assigning the i th iteration in each of the inner loops to the same processor, each processor always accesses the same rows of the arrays throughout the entire computation. Figure 1(b) shows the data accessed by each processor in the case where each processor is assigned a block of rows. In this way, no interprocessor communication or synchronization is necessary.

Due to characteristics found in typical data caches, it is not sufficient to just minimize sharing between processors. First, data are transferred in fixed-size units known as *cache lines*, which are typically 4 to 128 bytes long[16]. A computation is said to have *spatial locality* if it uses multiple words in a cache line before the line is displaced from the cache. While spatial locality is a consideration for both uni- and multiprocessors, *false sharing* is unique to multiprocessors. False sharing results when different processors use different data that happen to be co-located on the same cache line. Even if a processor re-uses a data item, the item may no longer be in the cache due to an intervening access by another processor to another word in the same cache line.

Assuming the FORTRAN convention that arrays are allocated in column-major order, there is a significant amount of false sharing

This research was supported in part by ARPA contracts DABT63-91-K-0003 and DABT63-94-C-0054, an NSF Young Investigator Award and fellowships from Digital Equipment Corporation's Western Research Laboratory and Intel Corporation.

In *Proceedings of Fifth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '95)*
Santa Barbara, CA, July 19–21, 1995

```

REAL A(N,N), B(N,N), C(N,N)
DO 30 time = 1, NSTEPS
  ...
  DO 10 J = 1, N
    DO 10 I = 1, N
      A(I,J) = B(I,J) + C(I,J)
10  CONTINUE
  DO 20 J = 2, N-1
    DO 20 I = 1, N
      A(I,J) = 0.333 * (A(I,J) + A(I,J-1) + A(I,J+1))
20  CONTINUE
  ...
30  CONTINUE

```

(a)

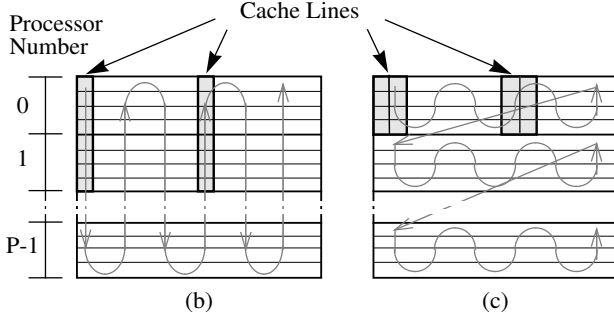


Figure 1: A simple example: (a) sample code, (b) original data mapping and (c) optimized data mapping. The light grey arrows show the memory layout order.

in our example, as shown in Figure 1(b). If the number of rows accessed by each processor is smaller than the number of words in a cache line, every cache line is shared by at least two processors. Each time one of these lines is accessed, unwanted data are brought into the cache. Also, when one processor writes part of the cache line, that line is invalidated in the other processor's cache. This particular combination of computation mapping and data layout will result in poor cache performance.

Another problematic characteristic of data caches is that they typically have a small set-associativity; that is, each memory location can only be cached in a small number of cache locations. *Conflict misses* occur whenever different memory locations contend for the same cache location. Since each processor only operates on a subset of the data, the addresses accessed by each processor may be distributed throughout the shared address space.

Consider what happens to the example in Figure 1(b) if the arrays are of size 1024×1024 and the target machine has a direct-mapped cache of size 64KB. Assuming that REALs are 4B long, the elements in every 16th column will map to the same cache location and cause conflict misses. This problem exists even if the caches are set-associative, given that existing caches usually only have a small degree of associativity.

As shown above, the cache performance of multiprocessor code depends on how the computation is distributed as well as how the data are laid out. Instead of simply obeying the data layout convention used by the input language (e.g. column-major in FORTRAN and row-major in C), we can improve the cache performance by customizing the data layout for the specific program. We observe

that multiprocessor cache performance problems can be minimized by making the data accessed by each processor contiguous in the shared address space, an example of which is shown in Figure 1(c). Such a layout enhances spatial locality, minimizes false sharing and also minimizes conflict misses.

The importance of optimizing memory subsystem performance for multiprocessors has also been confirmed by several studies of hand optimizations on real applications. Singh et al. explored performance issues on scalable shared address space architectures; they improved cache behavior by transforming two-dimensional arrays into four-dimensional arrays so that each processor's local data are contiguous in memory[28]. Torrellas et al.[30] and Eggers et al.[11, 12] also showed that improving spatial locality and reducing false sharing resulted in significant speedups for a set of programs on shared-memory machines. In summary, not only must we minimize sharing to achieve efficient parallelization, it is also important to optimize for the multi-word cache line and the small set associativity. The cache behavior depends on both the computation mapping and the data layout. Thus, besides choosing a good parallelization scheme and a good computation mapping, we may also wish to change the data structures in the program.

1.2 Overview

This paper presents a fully automatic compiler that translates sequential code to efficient parallel code on shared address space machines. Here we address the memory hierarchy optimization problems that are specific to multiprocessors; the algorithm described in the paper can be followed with techniques that improve locality on uniprocessor code[9, 13, 34]. Uniprocessor cache optimization techniques are outside the scope of this paper.

We have developed an integrated compiler algorithm that selects the proper loops to parallelize, assigns the computation to the processors, and changes the array data layout, all with the overall goal of improving the memory subsystem performance. While loop transformation is relatively well understood, data transformation is not. In this paper, we show that various well-known data layouts can be derived as a combination of two simple primitives: strip-mining and permutation. Both of these transforms have a direct analog in the theory of loop transformations[6, 35].

The techniques described in this paper are all implemented in the SUIF compiler system[33]. Our compiler takes sequential C or FORTRAN programs as input and generates optimized SPMD (Single Program Multiple Data) C code fully automatically. We ran our compiler over a set of sequential FORTRAN programs and measured the performance of our parallelized code on the DASH multiprocessor. This paper also includes measurements and performance analysis on these programs.

The rest of the paper is organized as follows. We first present an overview of our algorithm and the rationale of the design in Section 2. We then describe the two main steps of our algorithm in Sections 3 and Section 4. We compare our approach to related work in Section 5. In Section 6 we evaluate the effectiveness of the algorithm by applying the compiler to a number of benchmarks. Finally, we present our conclusions in Section 7.

2 Synopsis and Rationale of an Integrated Algorithm

As discussed in Section 1.1, memory hierarchy considerations permeate many aspects of the compilation process: how to parallelize the code, how to distribute the parallel computation across the processors and how to lay out the arrays in memory. By analyzing this complex problem carefully, we are able to partition the problem into the following two subproblems.

1. *How to obtain maximum parallelism while minimizing synchronization and true sharing.*

Our first goal is to find a parallelization scheme that incurs minimum synchronization and communication cost, without regard to the original layout of the data structures. This step of our algorithm determines how to assign the parallel computation to processors, and also what data are used by each processor. In this paper, we will refer to the former as *computation decomposition* and the latter as *data decomposition*.

We observe that efficient hand-parallelized codes synchronize infrequently and have little true sharing between processors[27]. Thus our algorithm is designed to find a parallelization scheme that requires no communication whenever possible. Our analysis starts with the model that all loop iterations and array elements are distributed. As the analysis proceeds, the algorithm groups together computation that uses the same data. Communication is introduced at the least executed parts of the code only to avoid collapsing all parallelizable computation onto one processor. In this way, the algorithm groups the computation and data into the largest possible number of (mostly) disjoint sets. By assigning one or more such sets of data and computation to processors, the compiler finds a parallelization scheme, a computation decomposition and a data decomposition that minimize communication.

2. *How to enhance spatial locality, reduce false sharing and reduce conflict misses in multiprocessor code.*

While the data decompositions generated in the above step have traditionally been used to manage data on distributed address space machines, this information is also very useful for shared address space machines. Once the code has been parallelized in the above manner, it is clear that we need only to improve locality among accesses to each set of data assigned to a processor. As discussed in Section 1.1, multiprocessors have especially poor cache performances because the processors often operate on disconnected regions of the address space. We can change this characteristic by making the data accessed by each processor contiguous, using the data decomposition information generated in the first step.

The algorithm we developed for this subproblem is rather straightforward. The algorithm derives the desired data layout by systematically applying two simple data transforms: strip-mining and permutation. The algorithm also applies several novel optimizations to improve the address calculations for transformed arrays.

3 Minimizing Synchronization and Communication

The problem of minimizing data communication is fundamental to all parallel machines, be they distributed or shared address space machines. A popular approach to this problem is to leave the responsibility to the user; the user specifies the data-to-processor mapping using a language such as HPF[17], and the compiler infers the computation mapping by using the owner-computes rule[18]. Recently a number of algorithms for finding data and/or computation decompositions automatically have been proposed[2, 4, 5, 7, 15, 25, 26]. In keeping with our observation that communication in efficient parallel programs is infrequent, our algorithm is unique in that it offers a simple procedure to find the largest available degree of parallelism that requires no major data communication[4]. We present a brief overview of the algorithm, and readers are referred to [4] for more details.

3.1 Representation

Our algorithm represents loop nests and arrays as multi-dimensional spaces. Computation and data decompositions are represented as a two-step mapping: we first map the loop nests and arrays onto a virtual processor space via affine transformations, and then map the virtual processors onto the physical processors of the target machine via one of the following folding functions: BLOCK, CYCLIC or BLOCK-CYCLIC.

The model used by our compiler represents a superset of the decompositions available to HPF programmers. The affine functions specify the array alignments in HPF. The rank of the linear transformation part of the affine function specifies the dimensionality of the processor space; this corresponds to the dimensions in the DISTRIBUTE statement that are not marked as “*”. The folding functions map directly to those used in the DISTRIBUTE statement in HPF. As the HPF notation is more familiar, we will use HPF in the examples in the rest of the paper.

3.2 Summary of Algorithm

The first step of the algorithm is to analyze each loop nest individually and restructure the loop via unimodular transformations to expose the largest number of outermost parallelizable loops[35]. This is just a preprocessing step, and we do not make any decision on which loops to parallelize yet.

The second step attempts to find affine mappings of data and computation onto the virtual processor space that maximize parallelism without requiring any major communication. A necessary and sufficient condition for a processor assignment to incur no communication cost is that each processor must access only data that are local to the processor. Let C_j be a function mapping iterations in loop nest j to the processors, and let D_x be a function mapping elements of array x to the processors. Let F_{jx} be a reference to array x in loop j . No communication is necessary iff

$$\forall F_{jx}, D_x(F_{jx}(i)) = C_j(i) \quad (1)$$

The algorithm tries to find decompositions that satisfy this equation. The objective is to maximize the rank of the linear transformations, as the rank corresponds to the degree of parallelism in the program.

Our algorithm tries to satisfy Equation 1 incrementally, starting with the constraints among the more frequently executed loops and

finally to the less frequently executed loops. If it is not possible to find non-trivial decompositions that satisfy the equation, we introduce communication. Read-only and seldom-written data can be replicated; otherwise we generate different data decompositions for different parts of the program. This simple, greedy approach tends to place communication in the least executed sections of the code.

The third step is to map the virtual processor space onto the physical processor space. At this point, the algorithm has mapped the computation to an n^d virtual processor space, where d is the degree of parallelism in the code, and n is the number of iterations in the loops (assuming that the number of iterations is the same across the loops). It is well known that parallelizing as many dimensions of loops as possible tends to decrease the communication to computation ratio. Thus, by default, our algorithm partitions the virtual processor space into d -dimensional blocks and assigns a block to each physical processor. If we observe that the computation of an iteration in a parallelized loop either decreases or increases with the iteration number, we choose a cyclic distribution scheme to enhance the load balance. We choose a block-cyclic scheme only when pipelining is used in parallelizing a loop and load balance is an issue.

Finally, we find the mapping of computation and data to the physical processors by composing the affine functions with the virtual processor folding function. Internally, the mappings are represented as a set of linear inequalities, even though we use the HPF notation here for expository purposes. The data decomposition is used by the data transformation phase, and the computation decomposition is used to generate the SPMD code.

The decomposition analysis must be performed across the entire program. This is a difficult problem since the compiler must map the decompositions across the procedure boundaries. The inter-procedural analysis must be able to handle array reshapes and array sections passed as parameters. Previously, our implementation was limited by procedure boundaries[4]. We have now implemented a prototype of an inter-procedural version of this algorithm that was used for the experiments described in Section 6.

3.3 Example

We now use the code shown in Figure 1(a) to illustrate our algorithm. Because of the data dependence carried by the `DO 20 J` loop, iterations of this loop must execute on the same processor in order for there to be no communication. Starting with this constraint and applying Equation 1, the compiler finds that the second dimension of arrays A, B and C must also be allocated to the same processor. Equation 1 is applied again to find that the `DO 10 J` must also run on the same processor. Finally, the folding function for this example is BLOCK as selected by default. The final data decompositions for the arrays are DISTRIBUTE(BLOCK, *).

4 Data Transformations

Given the data to processor mappings calculated by the first step of the algorithm, the second step restructures the arrays so that all the data accessed by the same processor are contiguous in the shared address space.

4.1 Data Transformation Model

To facilitate the design of our data layout algorithm, we have developed a data transformation model that is analogous to the well-known loop transformation theory[6, 35]. We represent an n -dimensional array as an n -dimensional polytope whose boundaries are given by the array bounds, and the interior integer points represent all the elements in the array. As with sequential loops, the ordering of the axes is significant. In the rest of the paper we assume the FORTRAN convention of column-major ordering by default, and for clarity the array dimensions are 0-based. This means that for an n -dimensional array with array bounds $d_1 \times d_2 \times \dots \times d_n$, the linearized address for array element $(i_1, i_2, \dots, i_n)$ is $((\dots((i_n \times d_{n-1} + i_{n-1}) \times d_{n-2} + i_{n-2}) \times \dots + i_3) \times d_2 + i_2) \times d_1 + i_1$. Below we introduce two primitive data transforms: *strip-mining* and *permutation*.

4.1.1 Strip-mining

Strip-mining an array dimension re-organizes the original data in the dimension as a two-dimensional structure. For example, strip-mining a one-dimensional d -element array with strip size b turns the array into a $b \times \lceil \frac{d}{b} \rceil$ array. Figure 2(a) shows the data in the original array, and Figure 2(b) shows the new indices in the strip-mined array. The first column of this strip-mined array is high-lighted in the figure. The number in the upper right corner of each square shows the linear address of the data item in the new array. The i th element in the original array now has coordinates $(i \bmod b, \lfloor \frac{i}{b} \rfloor)$ in the strip-mined array. Given that block sizes are positive, with the assumption that arrays are 0-based we can replace the floor operators in array access functions with integer division assuming truncation. The address of the element in the linear memory space is $\frac{i}{b} \times b + i \bmod b = i$. Strip-mining, on its own, does not change the layout of the data in memory. It must be combined with other transformations to have an effect.

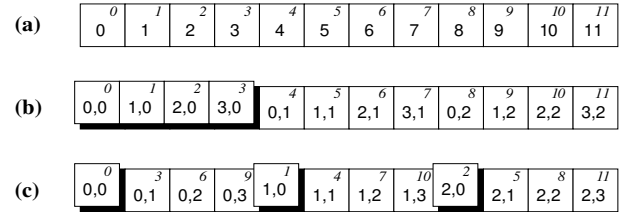


Figure 2: Array indices of (a) the original array, (b) the strip-mined array and (c) the final array. The numbers in the upper right corners show the linearized addresses of the data.

4.1.2 Permutation

A permutation transform T maps an n -dimensional array space to another n -dimensional space; that is, if $\vec{i}$ is the original array index vector, the transformed array indices $\vec{i}'$ is $\vec{i}' = T\vec{i}$. The array bounds must also be transformed similarly.

For example, an array transpose maps (i_1, i_2) to (i_2, i_1) . Using matrix notation this becomes

$$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} = \begin{bmatrix} i_2 \\ i_1 \end{bmatrix}$$

The result of transposing the array in Figure 2(b) is shown in Figure 2(c). Figure 2(c) shows the data in the original layout, and each item is labeled with its new indices in the transposed array in the center, and its new linearized address in the upper right corner. As high-lighted in the diagram, this example shows how a combination of strip-mining and permutation can make every fourth data element in a linear array contiguous to each other. This is used, for example, to make contiguous a processor’s share of data in a cyclically distributed array.

In theory, we can generalize permutations to other unimodular transforms. For example, rotating a two-dimensional array by 45 degrees makes data along a diagonal contiguous, which may be useful if a loop accesses the diagonal in consecutive iterations. There are two plausible ways of laying the data out in memory. The first is to embed the resulting parallelogram in the smallest enclosing rectilinear space, and the second is to simply place the diagonals consecutively, one after the other. The former has the advantage of simpler address calculation, and the latter has the advantage of more compact storage. We do not expect unimodular transforms other than permutations to be important in practice.

4.1.3 Legality

Unlike loop transformations, which must satisfy data dependences, data transforms are not limited by any ordering constraints. On the other hand, loop transforms have the advantage that they affect only one specific loop; performing an array data transform requires that all accesses to the array in the entire program use the new layout. Current programming languages like C and FORTRAN have features that can make these transformations difficult. The compiler cannot restructure an array unless it can guarantee that all possible accesses of the same data can be updated accordingly. For example, in FORTRAN, the storage for a common block array in one procedure can be re-used to form a completely different set of data structures in another procedure. In C, pointer arithmetic and type casting can prevent data transformations.

4.2 Algorithm

Given the data decompositions calculated by the computation decomposition phase, there are many equivalent memory layouts that make each processor’s data contiguous in the shared address space. Consider the example in Figure 1. Each processor accesses a two-dimensional block of data. There remains the question of whether the elements within the blocks, and the blocks themselves, should be organized in a column-major or row-major order. Our current implementation simply retains the original data layout as much as possible. That is, all the data accessed by the same processor maintain the original relative ordering. We expect this compilation phase to be followed by another algorithm that analyzes the computation executed by *each* processor and improves the cache performance by reordering data and operations on each processor.

Since general affine decompositions rarely occur in practice and the corresponding data transformations would result in complex array access functions, our current implementation requires that only a single array dimension can be mapped to one processor dimension.

Our algorithm applies the following procedure to each distributed array dimension. First, we apply strip-mining according to the type of distribution:

BLOCK. Strip-mine the dimension with strip size $\lceil \frac{d}{P} \rceil$, where d is

the size of the dimension, and P is the number of processors. The identifier of the processor owning the data is specified by the second of the strip-mined dimensions.

CYCLIC. Strip-mine the dimension with strip size P , where P is the number of processors. The identifier of the processor owning the data is specified by the first of the strip-mined dimensions.

BLOCK-CYCLIC. First strip-mine the dimension with the given block size b as the strip size; then further strip-mine the second of the strip-mined dimensions with strip size P , where P is the number of processors. The identifier of the processor owning the data is specified by the middle of the strip-mined dimensions.

Next, move the strip-mined dimension that identifies the processor to the rightmost position of the array index vector.

Figure 3 shows how our algorithm restructures several two-dimensional arrays according to the specified distribution. Figure 3(a) shows the intermediate array index calculations corresponding to the strip-mining step of the algorithm. Figure 3(b) shows the final array indices obtained by permuting the processor-defining dimension to the rightmost position of the array index function. Figure 3(c) shows an example of the new array layout in memory and Figure 3(d) shows the new dimensions of the restructured array. The figure high-lights the set of data belonging to the same processor; the new array indices and the linear addresses at the upper right corner indicate that they are contiguous in the address space.

This technique can be repeated for every distributed dimension of a multi-dimensional array. Each of the distributed dimensions contributes one processor-identifying dimension that is moved to the rightmost position. As discussed before, it does not matter how we order these higher dimensions. By not permuting those dimensions that do not identify processors, we retain the original relative ordering among data accessed by the same processor.

Finally, we make one minor local optimization. If the highest dimension of the array is distributed as BLOCK, no permutation is necessary since the processor dimension is already in the rightmost position; thus no strip-mining is necessary either since, as discussed above, strip-mining on its own does not change the data layout.

Note that HPF statements can also be used as input to the data transformation algorithm. If an array is aligned to a template which is then distributed, we must find the equivalent distribution on the array directly. We use the alignment function to map from the distributed template dimensions back to the corresponding array dimensions. Any offsets in the alignment statement are ignored.

4.3 Code Generation

The exact dimensions of a transformed array often depend on the number of processors, which may not known at compile time. For example, if P is the number of processors and d is the size of the dimension, the strip sizes used in CYCLIC and BLOCK distributions are P and $\lceil \frac{d}{P} \rceil$, respectively. Since our compiler outputs C code, and C does not support general multi-dimensional arrays with dynamic sizes, our compiler declares the array as a linear array and uses linearized addresses to access the array elements. As discussed above, strip-mining a d -element array dimension with strip size b produces a subarray of size $b \times \lceil \frac{d}{b} \rceil$. This total size can be greater than d , but is always less than $d + b - 1$. We can still allocate the

	ORIGINAL	(BLOCK, *)	(CYCLIC, *)	(BLOCK-CYCLIC, *)																																																																																																																																																																																																															
(a)	(i_1, i_2)	$\left(i_1 \bmod \lceil \frac{d_1}{P} \rceil, \frac{i_1}{\lceil \frac{d_1}{P} \rceil}, i_2\right)$	$(i_1 \bmod P, \frac{i_1}{P}, i_2)$	$(i_1 \bmod b, \frac{i_1}{b}, i_2)$																																																																																																																																																																																																															
(b)	(i_1, i_2)	$\left(i_1 \bmod \lceil \frac{d_1}{P} \rceil, i_2, \frac{i_1}{\lceil \frac{d_1}{P} \rceil}\right)$	$(\frac{i_1}{P}, i_2, i_1 \bmod P)$	$(i_1 \bmod b, \frac{i_1}{Pb}, i_2, \frac{i_1}{b} \bmod P)$																																																																																																																																																																																																															
(c)	i_2<table><tr><td>0</td><td>8</td><td>16</td><td>24</td></tr><tr><td>0,0</td><td>0,1</td><td>0,2</td><td>0,3</td></tr><tr><td>1</td><td>9</td><td>17</td><td>25</td></tr><tr><td>1,0</td><td>1,1</td><td>1,2</td><td>1,3</td></tr><tr><td>2</td><td>10</td><td>18</td><td>26</td></tr><tr><td>2,0</td><td>2,1</td><td>2,2</td><td>2,3</td></tr><tr><td>3</td><td>11</td><td>19</td><td>27</td></tr><tr><td>3,0</td><td>3,1</td><td>3,2</td><td>3,3</td></tr><tr><td>4</td><td>12</td><td>20</td><td>28</td></tr><tr><td>4,0</td><td>4,1</td><td>4,2</td><td>4,3</td></tr><tr><td>5</td><td>13</td><td>21</td><td>29</td></tr><tr><td>5,0</td><td>5,1</td><td>5,2</td><td>5,3</td></tr><tr><td>6</td><td>14</td><td>22</td><td>30</td></tr><tr><td>6,0</td><td>6,1</td><td>6,2</td><td>6,3</td></tr><tr><td>7</td><td>15</td><td>23</td><td>31</td></tr><tr><td>7,0</td><td>7,1</td><td>7,2</td><td>7,3</td></tr></table>	0	8	16	24	0,0	0,1	0,2	0,3	1	9	17	25	1,0	1,1	1,2	1,3	2	10	18	26	2,0	2,1	2,2	2,3	3	11	19	27	3,0	3,1	3,2	3,3	4	12	20	28	4,0	4,1	4,2	4,3	5	13	21	29	5,0	5,1	5,2	5,3	6	14	22	30	6,0	6,1	6,2	6,3	7	15	23	31	7,0	7,1	7,2	7,3	i_2<table><tr><td>0</td><td>4</td><td>8</td><td>12</td></tr><tr><td>0,0,0</td><td>0,1,0</td><td>0,2,0</td><td>0,3,0</td></tr><tr><td>1</td><td>5</td><td>9</td><td>13</td></tr><tr><td>1,0,0</td><td>1,1,0</td><td>1,2,0</td><td>1,3,0</td></tr><tr><td>2</td><td>6</td><td>10</td><td>14</td></tr><tr><td>2,0,0</td><td>2,1,0</td><td>2,2,0</td><td>2,3,0</td></tr><tr><td>3</td><td>7</td><td>11</td><td>15</td></tr><tr><td>3,0,0</td><td>3,1,0</td><td>3,2,0</td><td>3,3,0</td></tr><tr><td>16</td><td>20</td><td>24</td><td>28</td></tr><tr><td>0,0,1</td><td>0,1,1</td><td>0,2,1</td><td>0,3,1</td></tr><tr><td>17</td><td>21</td><td>25</td><td>29</td></tr><tr><td>1,0,1</td><td>1,1,1</td><td>1,2,1</td><td>1,3,1</td></tr><tr><td>2</td><td>6</td><td>10</td><td>14</td></tr><tr><td>2,0,1</td><td>2,1,1</td><td>2,2,1</td><td>2,3,1</td></tr><tr><td>18</td><td>22</td><td>26</td><td>30</td></tr><tr><td>2,0,1</td><td>2,1,1</td><td>2,2,1</td><td>2,3,1</td></tr><tr><td>3</td><td>7</td><td>11</td><td>15</td></tr><tr><td>3,0,1</td><td>3,1,0</td><td>3,2,0</td><td>3,3,0</td></tr><tr><td>19</td><td>23</td><td>27</td><td>31</td></tr><tr><td>3,0,1</td><td>3,1,1</td><td>3,2,1</td><td>3,3,1</td></tr></table>	0	4	8	12	0,0,0	0,1,0	0,2,0	0,3,0	1	5	9	13	1,0,0	1,1,0	1,2,0	1,3,0	2	6	10	14	2,0,0	2,1,0	2,2,0	2,3,0	3	7	11	15	3,0,0	3,1,0	3,2,0	3,3,0	16	20	24	28	0,0,1	0,1,1	0,2,1	0,3,1	17	21	25	29	1,0,1	1,1,1	1,2,1	1,3,1	2	6	10	14	2,0,1	2,1,1	2,2,1	2,3,1	18	22	26	30	2,0,1	2,1,1	2,2,1	2,3,1	3	7	11	15	3,0,1	3,1,0	3,2,0	3,3,0	19	23	27	31	3,0,1	3,1,1	3,2,1	3,3,1	i_2<table><tr><td>0</td><td>4</td><td>8</td><td>12</td></tr><tr><td>0,0,0</td><td>0,1,0</td><td>0,2,0</td><td>0,3,0</td></tr><tr><td>1</td><td>5</td><td>9</td><td>13</td></tr><tr><td>1,0,0</td><td>1,0,1,0</td><td>1,0,2,0</td><td>1,0,3,0</td></tr><tr><td>16</td><td>20</td><td>24</td><td>28</td></tr><tr><td>0,0,1</td><td>0,0,1,1</td><td>0,0,2,1</td><td>0,0,3,1</td></tr><tr><td>17</td><td>21</td><td>25</td><td>29</td></tr><tr><td>1,0,1</td><td>1,0,1,1</td><td>1,0,2,1</td><td>1,0,3,1</td></tr><tr><td>2</td><td>6</td><td>10</td><td>14</td></tr><tr><td>0,1,0,0</td><td>0,1,1,0</td><td>0,1,2,0</td><td>0,1,3,0</td></tr><tr><td>18</td><td>22</td><td>26</td><td>30</td></tr><tr><td>1,1,0,0</td><td>1,1,1,0</td><td>1,1,2,0</td><td>1,1,3,0</td></tr><tr><td>3</td><td>7</td><td>11</td><td>15</td></tr><tr><td>0,1,0,1</td><td>0,1,1,1</td><td>0,1,2,1</td><td>0,1,3,1</td></tr><tr><td>19</td><td>23</td><td>27</td><td>31</td></tr><tr><td>1,1,0,1</td><td>1,1,1,1</td><td>1,1,2,1</td><td>1,1,3,1</td></tr></table>	0	4	8	12	0,0,0	0,1,0	0,2,0	0,3,0	1	5	9	13	1,0,0	1,0,1,0	1,0,2,0	1,0,3,0	16	20	24	28	0,0,1	0,0,1,1	0,0,2,1	0,0,3,1	17	21	25	29	1,0,1	1,0,1,1	1,0,2,1	1,0,3,1	2	6	10	14	0,1,0,0	0,1,1,0	0,1,2,0	0,1,3,0	18	22	26	30	1,1,0,0	1,1,1,0	1,1,2,0	1,1,3,0	3	7	11	15	0,1,0,1	0,1,1,1	0,1,2,1	0,1,3,1	19	23	27	31	1,1,0,1	1,1,1,1	1,1,2,1	1,1,3,1
0	8	16	24																																																																																																																																																																																																																
0,0	0,1	0,2	0,3																																																																																																																																																																																																																
1	9	17	25																																																																																																																																																																																																																
1,0	1,1	1,2	1,3																																																																																																																																																																																																																
2	10	18	26																																																																																																																																																																																																																
2,0	2,1	2,2	2,3																																																																																																																																																																																																																
3	11	19	27																																																																																																																																																																																																																
3,0	3,1	3,2	3,3																																																																																																																																																																																																																
4	12	20	28																																																																																																																																																																																																																
4,0	4,1	4,2	4,3																																																																																																																																																																																																																
5	13	21	29																																																																																																																																																																																																																
5,0	5,1	5,2	5,3																																																																																																																																																																																																																
6	14	22	30																																																																																																																																																																																																																
6,0	6,1	6,2	6,3																																																																																																																																																																																																																
7	15	23	31																																																																																																																																																																																																																
7,0	7,1	7,2	7,3																																																																																																																																																																																																																
0	4	8	12																																																																																																																																																																																																																
0,0,0	0,1,0	0,2,0	0,3,0																																																																																																																																																																																																																
1	5	9	13																																																																																																																																																																																																																
1,0,0	1,1,0	1,2,0	1,3,0																																																																																																																																																																																																																
2	6	10	14																																																																																																																																																																																																																
2,0,0	2,1,0	2,2,0	2,3,0																																																																																																																																																																																																																
3	7	11	15																																																																																																																																																																																																																
3,0,0	3,1,0	3,2,0	3,3,0																																																																																																																																																																																																																
16	20	24	28																																																																																																																																																																																																																
0,0,1	0,1,1	0,2,1	0,3,1																																																																																																																																																																																																																
17	21	25	29																																																																																																																																																																																																																
1,0,1	1,1,1	1,2,1	1,3,1																																																																																																																																																																																																																
2	6	10	14																																																																																																																																																																																																																
2,0,1	2,1,1	2,2,1	2,3,1																																																																																																																																																																																																																
18	22	26	30																																																																																																																																																																																																																
2,0,1	2,1,1	2,2,1	2,3,1																																																																																																																																																																																																																
3	7	11	15																																																																																																																																																																																																																
3,0,1	3,1,0	3,2,0	3,3,0																																																																																																																																																																																																																
19	23	27	31																																																																																																																																																																																																																
3,0,1	3,1,1	3,2,1	3,3,1																																																																																																																																																																																																																
0	4	8	12																																																																																																																																																																																																																
0,0,0	0,1,0	0,2,0	0,3,0																																																																																																																																																																																																																
1	5	9	13																																																																																																																																																																																																																
1,0,0	1,0,1,0	1,0,2,0	1,0,3,0																																																																																																																																																																																																																
16	20	24	28																																																																																																																																																																																																																
0,0,1	0,0,1,1	0,0,2,1	0,0,3,1																																																																																																																																																																																																																
17	21	25	29																																																																																																																																																																																																																
1,0,1	1,0,1,1	1,0,2,1	1,0,3,1																																																																																																																																																																																																																
2	6	10	14																																																																																																																																																																																																																
0,1,0,0	0,1,1,0	0,1,2,0	0,1,3,0																																																																																																																																																																																																																
18	22	26	30																																																																																																																																																																																																																
1,1,0,0	1,1,1,0	1,1,2,0	1,1,3,0																																																																																																																																																																																																																
3	7	11	15																																																																																																																																																																																																																
0,1,0,1	0,1,1,1	0,1,2,1	0,1,3,1																																																																																																																																																																																																																
19	23	27	31																																																																																																																																																																																																																
1,1,0,1	1,1,1,1	1,1,2,1	1,1,3,1																																																																																																																																																																																																																
(d)	(d_1, d_2)	$(\lceil \frac{d_1}{P} \rceil, d_2, P)$	$(\lceil \frac{d_1}{P} \rceil, d_2, P)$	$(b, \lceil \frac{d_1}{Pb} \rceil, d_2, P)$																																																																																																																																																																																																															

Figure 3: Changing data layouts: (a) strip-mined array indices, (b) final array indices, (c) new indices in restructured array and (d) array bounds.

array statically provided that we can bound the value of the block size. If b_{max} is the largest possible block size, we simply need to add $b_{max} - 1$ elements to the original dimension.

Producing the correct array index functions for transformed arrays is rather straightforward. However, the modified index functions now contain modulo and division operations; if these operations are performed on every array access, the overhead will be much greater than any performance gained by improved cache behavior. Simple extensions to standard compiler techniques such as loop invariant removal and induction variable recognition can move some of the division and modulo operators out of inner loops[3]. We have developed an additional set of optimizations that exploit the fundamental properties of these operations[14], as well as the specialized knowledge the compiler has about these address calculations. The optimizations, described below, have proved to be important and effective.

Our first optimization takes advantage of the fact that a processor often addresses only elements within a single strip-mined partition of the array. For example, the parallelized SPMD code for the second loop nest in Figure 1(a) is shown below.

```

b = ceiling(N/P)
C distribute A(block,*)
REAL A(0:b-1,N,0:P-1)
DO 20 J = 2, 99
  DO 21 I = b*myid+1, min(b*myid+b, 100)
    A(mod(I-1,b), J, (I-1)/b) = ...
21 CONTINUE
20 CONTINUE

```

The compiler can determine that within the range $b*myid+1 \leq I \leq \min(b*myid+b, 100)$, the expression $(I-1)/b$ is always equal to myid. Also, within this range, the expression

$\text{mod}(I-1, b)$ is a linear expression. This information allows the compiler to produce the following optimized code:

```

idiv = myid
DO 20 J = 2, 99
  imod = 0
  DO 22 I = b*myid+1, min(b*myid+b, 100)
    A(imod, J, idiv) = ...
    imod = imod + 1
22 CONTINUE
20 CONTINUE

```

It is more difficult to eliminate modulo and division operations when the data accessed in a loop cross the boundaries of strip-mined partitions. In the case where only the first or last few iterations cross such a boundary, we simply peel off those iterations and apply the above optimization on the rest of the loop.

Finally, we have also developed a technique to optimize modulo and division operations that is akin to strength reduction. This optimization is applicable when we apply the modulo operation to affine expressions of the loop index; divisions sharing the same operands can also be optimized along with the modulo operations. In each iteration through the loop, we increment the modulo operand. Only when the result is found to exceed the modulus must we perform the modulo and the corresponding division operations. Consider the following example:

```

DO 20 J = a, b
  x = mod(4*J+c, 64)
  y = (4*J+c)/64
  ...
20 CONTINUE

```

Combining the optimization described with the additional information in this example that the modulus is a multiple of the stride, we obtain the following efficient code:

```

xst = mod(c, 4)
x = mod(4*a+c, 64)
y = (4*a+c)/64
DO 20 J = a, b
...
x = x+4
IF (x.ge.64) THEN
  x = xst
  y = y + 1
ENDIF
20 CONTINUE

```

5 Related Work and Comparison

Previous work on compiler algorithms for optimizing memory hierarchy performance has focused primarily on loop transformations. Unimodular loop transformations, loop fusion and loop nest blocking restructure computation to increase uniprocessor cache re-use[9, 13, 34]. Copying data into contiguous regions has been studied as a means for reducing cache interference[23, 29].

Several researchers have proposed algorithms to transform computation and data layouts to improve memory system performance[10, 20]. The same optimizations are intended to change the data access patterns to improve locality on both uniprocessors and shared address space multiprocessors. For the multiprocessor case, they assume that decision of which loops to parallelize has already been determined. In contrast, the algorithm described in this paper concentrates directly on multiprocessor memory system performance. We globally analyze the program and explicitly determine which loops to parallelize so that data are re-used by the same processor as much as possible. Our experimental results show that there is often a choice of parallelization across loop nests, and that this decision significantly impacts the performance of the resulting program.

The scope of the data transformations used in the previous work are array permutations only; they do not consider strip-mining. By using strip-mining in combination with permutation, our compiler is able to optimize spatial locality by making the data used by each processor contiguous in the shared address space. This means, for example, that our compiler can achieve good cache performance by creating cyclic and multi-dimensional blocked distributions.

Previous approaches use search-based algorithms to select a combination of data and computation transformations that result in good cache performance. Instead we partition the problem into two well-defined subproblems. The first step minimizes communication and synchronization without regard to the data layout. The second step then simply makes the data accessed by each processor contiguous. After the compiler performs the optimizations described in this paper, code optimizations for uniprocessor cache performance can then be applied.

Compile-time data transformations have also been used to eliminate false-sharing in explicitly parallel C code[19]. The domain of that work is quite different from ours; we consider both data and computation transformations, and the code is parallelized automatically. Their compiler statically analyzes the program to determine the data accessed by each processor, and then try to group the data together. Two different transformations are used to aggregate the data. First, their compiler turn groups of vectors that are accessed by different processors into an array of structures. Each structure contains the aggregated data accessed by a single processor. Second, their compiler moves shared data into memory that is allocated local to each processor. References to the original data structures

are replaced with pointers to the newly allocated per-processor data structures. Lastly, their compiler can also pad data structures that have no locality (e.g. locks) to avoid false-sharing.

6 Experimental Results

All the algorithms described in this paper have been implemented in the SUIF compiler system[33]. To evaluate the effectiveness of our proposed algorithm, we ran our compiler over a set of programs, ran our compiler-generated code on the Stanford DASH multiprocessor[24] and compared our results to those obtained without using our techniques.

6.1 Experimental Setup

The inputs to the SUIF compiler are sequential FORTRAN or C programs. The output is a parallelized C program that contains calls to a portable run-time library. The C code is then compiled on the parallel machine using the native C compiler.

Our target machine is the DASH multiprocessor. DASH has a cache-coherent NUMA architecture. The machine we used for our experiments consists of 32 processors, organized into 8 clusters of 4 processors each. Each processor is a 33MHz MIPS R3000, and has a 64KB first-level cache and a 256KB second-level cache. Both the first- and second-level caches are direct-mapped and have 16B lines. Each cluster has 28MB of main memory. A directory-based protocol is used to maintain cache coherence across clusters. It takes a processor 1 cycle to retrieve data from its first-level cache, about 10 cycles from its second-level cache, 30 cycles from its local memory and 100-130 cycles from a remote memory. The DASH operating system allocates memory to clusters at the page level. The page size is 4KB and pages are allocated to the first cluster that touches the page. We compiled the C programs produced by SUIF using gcc version 2.5.8 at optimization level -O3.

To focus on the memory hierarchy issues, our benchmark suite includes only those programs that have a significant amount of parallelism. Several of these programs were identified as having memory performance problems in a simulation study[31]. We compiled each program under each of the methods described below, and plot the speed up of the parallelized code on DASH. All speedups are calculated over the best sequential version of each program.

BASE. We compiled the programs with the basic parallelizer in the original SUIF system. This parallelizer has capabilities similar to traditional shared-memory compilers such as KAP[22]. It has a loop optimizer that applies unimodular transformations to one loop at a time to expose outermost loop parallelism and to improve data locality among the accesses within the loop[34, 35].

COMP DECOMP. We first applied the basic parallelizer to analyze the individual loops, then applied the algorithm in Section 3 to find computation decompositions (and the corresponding data decompositions) that minimize communication across processors. These computation decompositions are passed to a code generator which schedules the parallel loops and inserts calls to the run-time library. The code generator also takes advantage of the information to optimize the synchronization in the program[32]. The data layouts are left unchanged and are stored according to the FORTRAN convention.

COMP DECOMP + DATA TRANSFORM. Here, we used the optimizations in the base compiler as well as all the techniques described in this paper. Given the data decompositions calculated during the computation decomposition phase, the compiler reorganizes the arrays in the parallelized code to improve spatial locality, as described in section 4.

6.2 Evaluation

In this section, we present the performance results for each of the benchmarks in our suite. We briefly describe the programs and discuss the opportunities for optimization.

6.2.1 Vpenta

Vpenta is one of the kernels in nasa7, a program in the SPEC92 floating-point benchmark suite. This kernel simultaneously inverts three pentadiagonal matrices. The performance results are shown in Figure 4. The base compiler interchanges the loops in the original code so that the outer loop is parallelizable and the inner loop carries spatial locality. Without such optimizations, the program would not even get the slight speedup obtained with the base compiler.

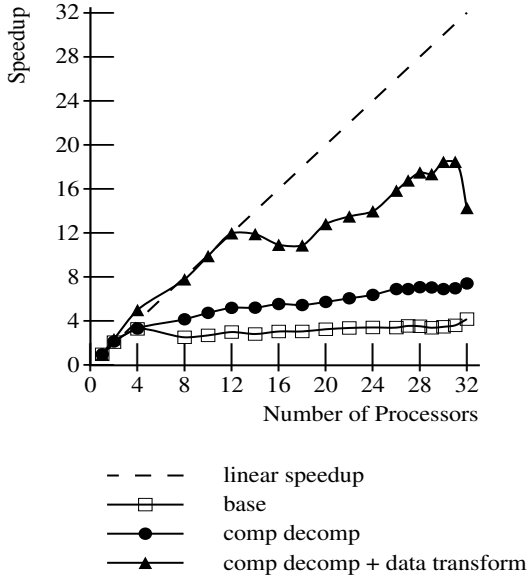


Figure 4: Vpenta Speedups

For this particular program, the base compiler’s parallelization scheme is the same as the results from the global analysis in our computation decomposition algorithm. However, since the compiler can determine that each processor accesses exactly the same partition of the arrays across the loops, the code generator can eliminate barriers between some of the loops. This accounts for the slight increase in performance of the computation decomposition version over the base compiler.

This program operates on a set of two-dimensional and three-dimensional arrays. Each processor accesses a block of columns for the two-dimensional arrays, thus no data reorganization is necessary for these arrays. However, each plane of the three-dimensional array is partitioned into blocks of rows, each of which is accessed by a

different processor. This presents an opportunity for our compiler to change the data layout and make the data accessed contiguous on each processor. With the improved data layout, the program finally runs with a decent speedup. We observe that the performance dips slightly when there are about 16 processors, and drops significantly when there are 32 processors. This performance degradation is due to increased cache conflicts among accesses within the same processor. Further data and computation optimizations that focus on operations on the same processor would be useful.

6.2.2 LU Decomposition

Our next program is LU decomposition without pivoting. The code is shown in Figure 5 and the speedups for each version of LU decomposition are displayed in Figure 6 for two different data set sizes (256×256 and 1024×1024).

```

DOUBLE PRECISION A(N,N)
DO 10 I1 = 1,N
DO 10 I2 = I1+1, N
  A(I2,I1) = A(I2,I1) / A(I1,I1)
  DO 10 I3 = I1+1, N
    A(I2,I3) = A(I2,I3) - A(I2,I1)*A(I1,I3)
10 CONTINUE

```

Figure 5: LU Decomposition Code

The base compiler identifies the second loop as the outermost parallelizable loop nest, and distributes its iterations uniformly across processors in a block fashion. As the number of iterations in this parallel loop varies with the index of the outer sequential loop, each processor accesses different data each time through the outer loop. A barrier is placed after the distributed loop and is used to synchronize between iterations of the outer sequential loop.

The computation decomposition algorithm minimizes true-sharing by assigning all operations on the same column of data to the same processor. For load balance, the columns and operations on the columns are distributed across the processor in a cyclic manner. By fixing the assignment of computation to processors, the compiler replaces the barriers that followed each execution of the parallel loop by locks. Even though this version has good load balance, good data re-use and inexpensive synchronization, the local data accessed by each processor are scattered in the shared address space, increasing chances of interference in the cache between columns of the array. The interference is highly sensitive to the array size and the number of processors; the effect of the latter can be seen in Figure 6. This interference effect can be especially pronounced if the array size and the number of processors are both powers of 2. For example, for the 1024×1024 matrix, every 8th column maps to the same location in DASH’s direct-mapped 64K cache. The speedup for 31 processors is 5 times better than for 32 processors.

The data transformation algorithm restructures the columns of the array so that each processor’s cyclic columns are made into a contiguous region. After restructuring, the performance stabilizes and is consistently high. In this case the compiler is able to take advantage of inexpensive synchronization and data re-use without incurring the cost of poor cache behavior. Speedups become super-linear in some cases due to the fact that once the data are partitioned among enough processors, each processor’s working set will fit into local memory.

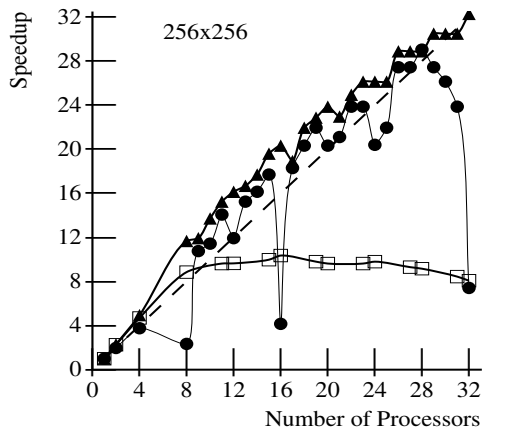


Figure 6: LU Decomposition Speedups

6.2.3 Five-Point Stencil

The code for our next example, a five-point stencil, is shown in Figure 7. Figure 8 shows the resulting speedups for each version of the code. The base compiler simply distributes the outermost parallel loop across the processors, and each processor updates a block of array columns. The values of the boundary elements are exchanged in each time step.

The computation decomposition algorithm assigns two-dimensional blocks to each processor, since this mapping has a better computation to communication ratio than a one-dimensional mapping. However, without also changing the data layout, the performance is worse than the base version because now each processor's partition is non-contiguous (in Figure 8, the number of processors in each of the two dimensions is also shown under the total number of processors).

After the data transformation is applied, the program has good spatial locality as well as less communication, and thus we achieve a speedup of 29 on 32 processors. Note that the performance is very

sensitive to the number of processors. This is due to the fact that each DASH cluster has 4 processors and the amount of communication across clusters differs significantly for different two-dimensional mappings.

```

REAL A(N,N), B(N,N)
C Initialize B
...
C Calculate Stencil
DO 30 time = 1, NSTEPS
...
DO 10 I1 = 1, N
DO 10 I2 = 2, N
A(I2, I1) = .20 * (B(I2, I1) + B(I2-1, I1) +
B(I2+1, I1) + B(I2, I1-1) + B(I2, I1+1))
10 CONTINUE
...
30 CONTINUE

```

Figure 7: Five-Point Stencil Code

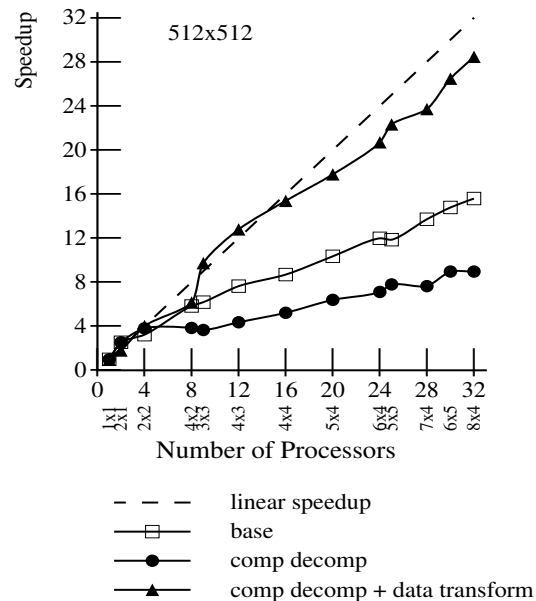


Figure 8: Five-Point Stencil Speedups

6.2.4 ADI Integration

ADI integration is a stencil computation used for solving partial differential equations. The computation in ADI has two phases — the first phase sweeps along the columns of the arrays and the second phase sweeps along the rows. Two representative loops of the code are shown in Figure 9. Figure 10 shows the speedups for each version of ADI Integration on two different data set sizes (256×256 and 1024×1024).

Given that the base compiler analyzes each loop separately, it makes the logical decision to parallelize and distribute first the column sweeps, then the row sweeps across the processors. This

```

REAL A(N,N), B(N,N), X(N,N)
...
DO 30 time = 1, NSTEPS
C Column Sweep
DO 10 I1 = 1, N
DO 10 I2 = 2, N
X(I2, I1) = X(I2, I1) - X(I2-1, I1) * A(I2, I1) / B(I2-1, I1)
B(I2, I1) = B(I2, I1) - A(I2, I1) * A(I2, I1) / B(I2-1, I1)
10 CONTINUE
...
C Row Sweep
DO 20 I1 = 2, N
DO 20 I2 = 1, N
X(I2, I1) = X(I2, I1) - X(I2, I1-1) * A(I2, I1) / B(I2, I1-1)
B(I2, I1) = B(I2, I1) - A(I2, I1) * A(I2, I1) / B(I2, I1-1)
20 CONTINUE
...
30 CONTINUE

```

Figure 9: ADI Integration Code

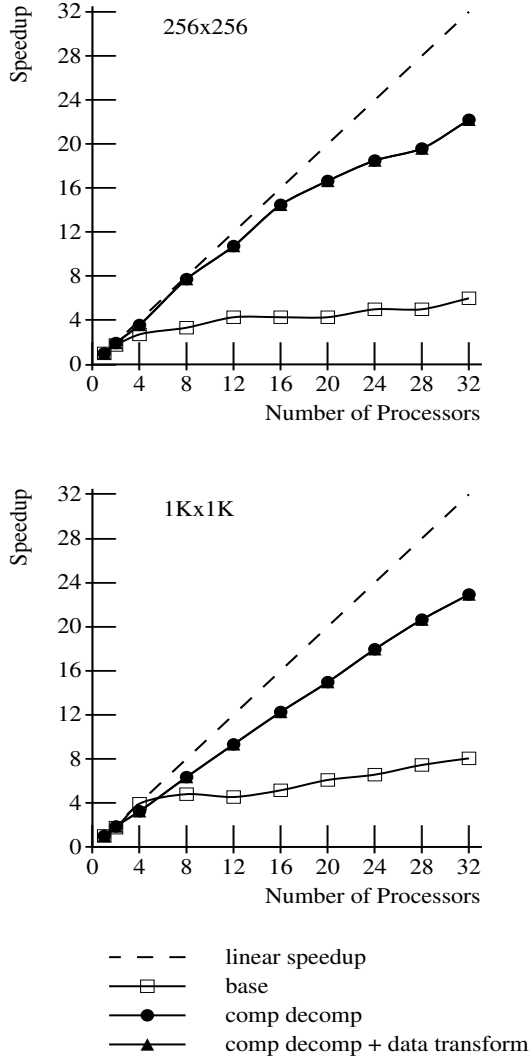


Figure 10: ADI Integration Speedups

base compiler retains the spatial locality in each of the loop nests, and inserts only one barrier at the end of each two-deep loop nest. Unfortunately, this means each processor accesses very different data in different parts of the algorithm. Furthermore, while data accessed by a processor in column sweeps are contiguous, the data accessed in row sweeps are distributed across the address space. As a result of the high miss rates and high memory access costs, the performance of the base version of ADI is rather poor.

By analyzing across the loops in the program, the computation decomposition algorithm finds a static block column-wise distribution. This version of the program exploits doall parallelism in the first phase of ADI, switching to doall/pipeline parallelism the second half of the computation to minimize true-sharing communication[4, 18]. Loops enclosed within the doacross loop are tiled to increase the granularity of pipelining, thus reducing synchronization overhead. The optimized version of ADI achieves a speedup of 23 on 32 processors. Since each processor's data are already contiguous, no data transformations are needed for this example.

6.2.5 Erlebacher

Erlebacher is a 600-line FORTRAN benchmark from ICASE that performs three-dimensional tridiagonal solves. It includes a number of fully parallel computations, interleaved with multi-dimensional reductions and computational wavefronts in all three dimensions caused by forward and backward substitutions. Partial derivatives are computed in all three dimensions with three-dimensional arrays. Figure 11 shows the resulting speedups for each version of Erlebacher.

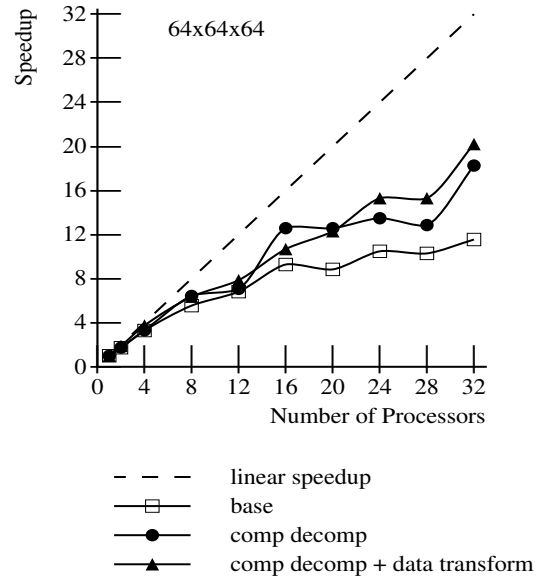


Figure 11: Erlebacher Speedups

The base-line version always parallelizes the outermost parallel loop. This strategy yields local accesses in the first two phases of Erlebacher when computing partial derivatives in the X and Y dimensions, but ends up causing non-local accesses in the Z dimension.

The computation decomposition algorithm improves the performance of Erlebacher slightly over the base-line version. It finds a computation decomposition so that no non-local accesses are needed in the Z dimension. The major data structures in the program are the input array and DUX, DUY and DUZ which are used to store the partial derivatives in the X , Y and Z dimensions, respectively. Since it is only written once, the input array is replicated. Each processor accesses a block of columns for arrays DUX and DUY, and a block of rows for array DUZ. Thus in this version of the program, DUZ has poor spatial locality. The data transformation phase of the compiler restructures DUZ so that local references are contiguous in memory. Because two-thirds of the program is perfectly parallel with all local accesses, the optimizations only realize a modest performance improvement.

6.2.6 Swm256

Swm256 is a 500-line program from the SPEC92 benchmark suite. It performs a two-dimensional stencil computation that applies finite-difference methods to solve shallow-water equations. The speedups for swm256 are shown in Figure 12.

Swm256 is highly data-parallel. Our base compiler is able to achieve good speedups by parallelizing the outermost parallel loop in all the frequently executed loop nests. The decomposition phase discovers that it can, in fact, parallelize both of the loops in the 2-deep loop nests in the program, without incurring any major data reorganization. The compiler chooses to exploit parallelism in both dimensions simultaneously, in an attempt to minimize the communication to computation ratio. Thus, the computation decomposition algorithm assigns two-dimensional blocks to each processor. However, the data accessed by each processor are scattered, causing poor cache performance. Fortunately, when we apply both the computation and data decomposition algorithm to the program, the program regains the performance lost and is slightly better than that obtained with the base compiler.

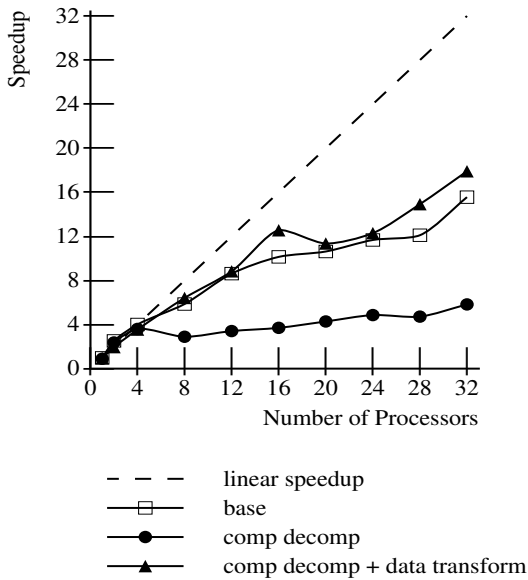


Figure 12: Swm256 Speedups

6.2.7 Tomcatv

Tomcatv is a 200-line mesh generation program from the SPEC92 floating-point benchmark suite. Figure 13 shows the resulting speedups for each version of tomcatv.

Tomcatv contains several loop nests that have dependences across the rows of the arrays and other loop nests that have no dependences. Since the base version always parallelizes the outermost parallel loop, each processor accesses a block of array columns in the loop nests with no dependences. However, in the loop nests with row dependences, each processor accesses a block of array rows. As a result, there is little opportunity for data re-use across loop nests. Also, there is poor cache performance in the row-dependent loop nests because the data accessed by each processor is not contiguous in the shared address space.

The computation decomposition pass of the compiler selects a computation decomposition so that each processor always accesses a block of rows. The row-dependent loop nests still execute completely in parallel. This version of tomcatv exhibits good temporal locality; however, the speedups are still poor due to poor cache behavior. After transforming the data to make each processor's rows contiguous, the cache performance improves. Whereas the maximum speedup achieved by the base version is 5, the fully optimized tomcatv achieves a speedup of 18.

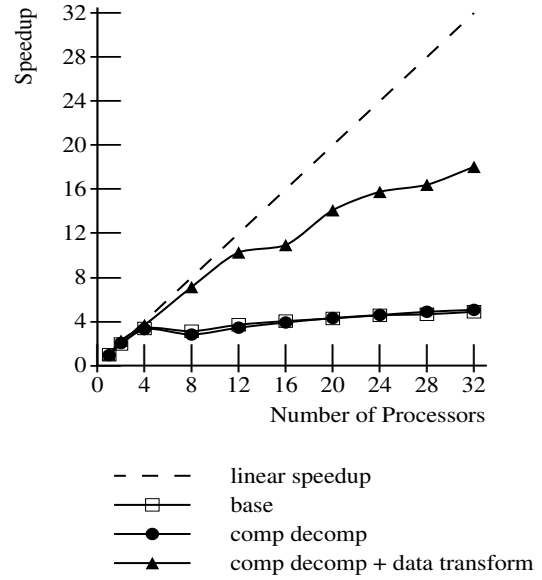


Figure 13: Tomcatv Speedups

6.3 Summary of Results

A summary of the results is presented in Table 1. For each program we compare the speedups on 32 processors obtained with the base compiler against the speedups obtained with all the optimizations turned on. We also indicate whether computation decomposition and data decomposition optimizations are critical to the improved performance. Finally, we list the data decompositions found for the major arrays in the program. Unless otherwise noted, the other arrays in the program were aligned with the listed array of the same dimensionality.

Program	Speedups (32 proc)		Critical Technique		Data Decompositions
	Base	Fully Optimized	Comp Decomp	Data Transform	
vpenta	4.2	14.3	✓	✓	F(*, BLOCK, *) A(*, BLOCK)
LU (1Kx1K)	19.5	33.5	✓	✓	A(*, CYCLIC)
stencil (512x512)	15.6	28.5	✓	✓	A(BLOCK,BLOCK)
ADI (1Kx1K)	8.0	22.9	✓		A(*,BLOCK)
erlebacher	11.6	20.2	✓	✓	DUX(*,*,BLOCK) DUY(*,*,BLOCK) DUZ(*,BLOCK,*)
swm256	15.6	17.9			P(BLOCK,BLOCK)
tomcatv	4.9	18.0	✓	✓	AA(BLOCK,*)

Table 1: Summary of Experimental Results

Our experimental results demonstrate that there is a need for memory optimizations on shared address space machines. The programs in our application suite are all highly parallelizable, but their speedups on a 32-processor machine are rather moderate, ranging from 4 to 20. Our compiler finds many opportunities for improvement; the data and computation decompositions are often different from the conventional or that obtained via local analysis. Finally, the results show that our algorithm is effective. The same set of programs now achieve 14 to 34-fold speedup on a 32-processor machine.

7 Conclusions

Even though shared address space machines have hardware support for coherence, getting good performance on these machines requires programmers to pay special attention to the memory hierarchy. Today, expert users restructure their codes and change their data structures manually to improve a program's locality of reference. This paper demonstrates that this optimization process can be automated. We have developed the first compiler system that fully automatically parallelizes sequential programs and changes the original array layouts. Our experimental results show that our algorithm can dramatically improve the parallel performance of shared address space machines.

The concepts described in this paper are useful for purposes other than translating sequential code to shared memory multiprocessors. Our algorithm to determine how to parallelize and distribute the computation and data is useful also to distributed address space machines. Our data transformation framework, consisting of the strip-mining and permuting primitives, is applicable to layout optimization for uniprocessors. Finally, our data transformation algorithm can also apply to HPF programs. While HPF directives are originally intended for distributed address space machines, our algorithm uses the information to make data accessed by each processor contiguous in the shared address space. In this way, the compiler achieves locality of reference, while taking advantage of the cache hardware to provide memory management and coherence functions.

Acknowledgements

The authors wish to thank Chau-Wen Tseng for his helpful discussions on this project and for his implementation of the synchronization optimization pass. Chris Wilson provided invaluable help with the experiments and suggested the strength reduction optimization on modulo and division operations. We also thank all the members of the Stanford SUIF compiler group for building and maintaining the infrastructure that was used to implement this work. We are also grateful to Dave Nakahira and the other members of the DASH group for maintaining the hardware platform we used for our experiments.

References

- [1] A. Agarwal, D. Chaiken, G. D'Souza, K. Johnson, and D. Kranz et. al. The MIT Alewife machine: A large-scale distributed memory multiprocessor. In *Scalable Shared Memory Multiprocessors*. Kluwer Academic Publishers, 1991.
- [2] A. Agarwal, D. Kranz, and V. Natarajan. Automatic partitioning of parallel loops for cache-coherent multiprocessors. In *Proceedings of the 1993 International Conference on Parallel Processing*, St. Charles, IL, August 1993.
- [3] A. V. Aho, R. Sethi, and J. D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Reading, MA, second edition, 1986.
- [4] J. M. Anderson and M. S. Lam. Global optimizations for parallelism and locality on scalable parallel machines. In *Proceedings of the SIGPLAN '93 Conference on Programming Language Design and Implementation*, pages 112–125, Albuquerque, NM, June 1993.
- [5] B. Appelbe and B. Lakshmanan. Optimizing parallel programs using affinity regions. In *Proceedings of the 1993 International Conference on Parallel Processing*, pages 246–249, St. Charles, IL, August 1993.
- [6] U. Banerjee, R. Eigenmann, A. Nicolau, and D. Padua. Automatic program parallelization. *Proceedings of the IEEE*, 81(2):211–243, February 1993.

- [7] B. Bixby, K. Kennedy, and U. Kremer. Automatic data layout using 0-1 integer programming. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT)*, pages 111–122, Montreal, Canada, August 1994.
- [8] W. J. Bolosky and M. L. Scott. False sharing and its effect on shared memory performance. In *Proceedings of the USENIX Symposium on Experiences with Distributed and Multiprocessor Systems (SEDS IV)*, pages 57–71, San Diego, CA, September 1993.
- [9] S. Carr, K. S. McKinley, and C.-W. Tseng. Compiler optimizations for improving data locality. In *Proceedings of the Sixth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-VI)*, pages 252–262, San Jose, CA, October 1994.
- [10] M. Cierniak and W. Li. Unifying data and control transformations for distributed shared memory machines. Technical Report TR-542, Department of Computer Science, University of Rochester, November 1994.
- [11] S. J. Eggers and T. E. Jeremiassen. Eliminating false sharing. In *Proceedings of the 1991 International Conference on Parallel Processing*, pages 377–381, St. Charles, IL, August 1991.
- [12] S. J. Eggers and R. H. Katz. The effect of sharing on the cache and bus performance of parallel programs. In *Proceedings of the Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-III)*, pages 257–270, Boston, MA, April 1989.
- [13] D. Gannon, W. Jalby, and K. Gallivan. Strategies for cache and local memory management by global program transformation. *Journal of Parallel and Distributed Computing*, 5(5):587–616, October 1988.
- [14] R. L. Graham, D. E. Knuth, and O. Patashnik. *Concrete Mathematics*. Addison-Wesley, Reading, MA, 1989.
- [15] M. Gupta and P. Banerjee. Demonstration of automatic data partitioning techniques for parallelizing compilers on multi-computers. *IEEE Transactions on Parallel and Distributed Systems*, 3(2):179–193, March 1992.
- [16] J. L. Hennessy and D. A. Patterson. *Computer Architecture A Quantitative Approach*. Morgan Kaufmann Publishers, San Mateo, CA, 1990.
- [17] High Performance Fortran Forum. High Performance Fortran language specification. *Scientific Programming*, 2(1-2):1–170, 1993.
- [18] S. Hiranandani, K. Kennedy, and C.-W. Tseng. Compiling Fortran D for MIMD distributed-memory machines. *Communications of the ACM*, 35(8):66–80, August 1992.
- [19] T. E. Jeremiassen and S. J. Eggers. Reducing false sharing on shared memory multiprocessors through compile time data transformations. Technical Report UW-CSE-94-09-05, Department of Computer Science and Engineering, University of Washington, September 1994.
- [20] Y. Ju and H. Dietz. Reduction of cache coherence overhead by compiler data layout and loop transformation. In U. Banerjee, D. Gelernter, A. Nicolau, and D. Padua, editors, *Languages and Compilers for Parallel Computing, Fourth International Workshop*, pages 344–358, Santa Clara, CA, August 1991. Springer-Verlag.
- [21] Kendall Square Research, Waltham, MA. *KSR1 Principles of Operation*, revision 6.0 edition, October 1992.
- [22] Kuck & Associates, Inc. *KAP User's Guide*. Champaign, IL 61820, 1988.
- [23] M. S. Lam, E. E. Rothberg, and M. E. Wolf. The cache performance and optimizations of blocked algorithms. In *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*, pages 63–74, Santa Clara, CA, April 1991.
- [24] D. Lenoski, J. Laudon, T. Joe, D. Nakahira, L. Stevens, A. Gupta, and J. Hennessy. The DASH prototype: Implementation and performance. In *Proceedings of the 19th International Symposium on Computer Architecture*, pages 92–105, Gold Coast, Australia, May 1992.
- [25] J. Li and M. Chen. The data alignment phase in compiling programs for distributed-memory machines. *Journal of Parallel and Distributed Computing*, 13(2):213–221, October 1991.
- [26] T. J. Sheffler, R. Schreiber, J. R. Gilbert, and S. Chatterjee. Aligning parallel arrays to reduce communication. In *Frontiers '95: The 5th Symposium on the Frontiers of Massively Parallel Computation*, pages 324–331, McLean, VA, February 1995.
- [27] J. P. Singh, W.-D. Weber, and A. Gupta. SPLASH: Stanford parallel applications for shared-memory. *Computer Architecture News*, 20(1):5–44, March 1992.
- [28] J.P. Singh, T. Joe, A. Gupta, and J. L. Hennessy. An empirical comparison of the Kendall Square Research KSR-1 and Stanford DASH multiprocessors. In *Proceedings of Supercomputing '93*, pages 214–225, Portland, OR, November 1993.
- [29] O. Temam, E. D. Granston, and W. Jalby. To copy or not to copy: A compile-time technique for assessing when data copying should be used to eliminate cache conflicts. In *Proceedings of Supercomputing '93*, pages 410–419, Portland, OR, November 1993.
- [30] J. Torrellas, M. S. Lam, and J. L. Hennessy. Shared data placement optimizations to reduce multiprocessor cache miss rates. In *Proceedings of the 1990 International Conference on Parallel Processing*, pages 266–270, St. Charles, IL, August 1990.
- [31] E. Torrie, C.-W. Tseng, M. Martonosi, and M. W. Hall. Evaluating the impact of advanced memory systems on compiler-parallelized codes. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT)*, June 1995.
- [32] C.-W. Tseng. Compiler optimizations for eliminating barrier synchronization. In *Proceedings of the Fifth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, July 1995.
- [33] R. P. Wilson, R. S. French, C. S. Wilson, S. P. Amarasinghe, J. M. Anderson, S. W. K. Tjiang, S.-W. Liao, C.-W. Tseng, M. W. Hall, M. S. Lam, and J. L. Hennessy. SUIF: An infrastructure for research on parallelizing and optimizing compilers. *ACM SIGPLAN Notices*, 29(12):31–37, December 1994.
- [34] M. E. Wolf and M. S. Lam. A data locality optimizing algorithm. In *Proceedings of the SIGPLAN '91 Conference on Programming Language Design and Implementation*, pages 30–44, Toronto, Canada, June 1991.
- [35] M. E. Wolf and M. S. Lam. A loop transformation theory and an algorithm to maximize parallelism. *IEEE Transactions on Parallel and Distributed Systems*, 2(4):452–471, October 1991.